UPN

UNIVERSIDAD
PRIVADA
DEL NORTE

SESIÓN 05:
REQUERIMIENTOS DE SISTEMAS.
HISTORIAS DE USUARIO.
DIAGRAMAS UML

Docente del Curso

INTERACCION MOTIVACIONAL

Reflexión

Puedes ver el siguiente video:

https://www.youtube.com/watch?v=Zi9E1aUO_1U&ab_channel=FattoConsultor%C3%ADaySistemas

Logro de Aprendizaje

Al terminar la sesión el estudiante desarrolla y expone casos de la programación orientada a objetos implementando Requerimientos de sistemas en Historias de usuario y representándolo en Diagramas UML relacionando con el lenguaje de programación de Java y con la asesoría brindada por el docente.

Fundamentos de los Requerimientos de Sistema en Desarrollo de Software

Definicion

Los requerimientos de sistema describen qué debe hacer un sistema de software para satisfacer las necesidades de los usuarios y del negocio.

Importancia

Son cruciales para asegurar que el sistema sistema se desarrolla de acuerdo con las las expectativas y para evitar errores y retrabajos.

Identificación y Documentación de Requerimientos Funcionales y No Funcionales

1. Los requerimientos funcionales describen las funciones específicas que debe realizar el sistema.

2. Los requerimientos no funcionales describen aspectos como la seguridad, el rendimiento, rendimiento, la usabilidad y usabilidad y la escalabilidad.

3. Fundamental documentar documentar los requerimientos de forma forma clara, concisa y detallada.

Historias de Usuario: Capturando las Necesidades del Cliente

Enfoque

Las historias de usuario describen las necesidades necesidades del cliente desde su punto de vista.

Formato

Tienen una estructura simple: "Como un [tipo de de usuario], quiero [accionar], para que [resultado]."

Ejemplo

Como un usuario registrado, quiero poder iniciar sesión en la sesión en la aplicación, para que pueda acceder a mis datos datos personales.

Estructura y Componentes de una Historia de Usuario Efectiva

Rol

Identifica al usuario que que realiza la acción.

Acción

Describe la acción que el que el usuario desea realizar.

Beneficio

Explica el resultado positivo que se obtiene obtiene al realizar la acción.

Introducción a UML: Conceptos Básicos y Tipos de Diagramas

Lenguaje Visual

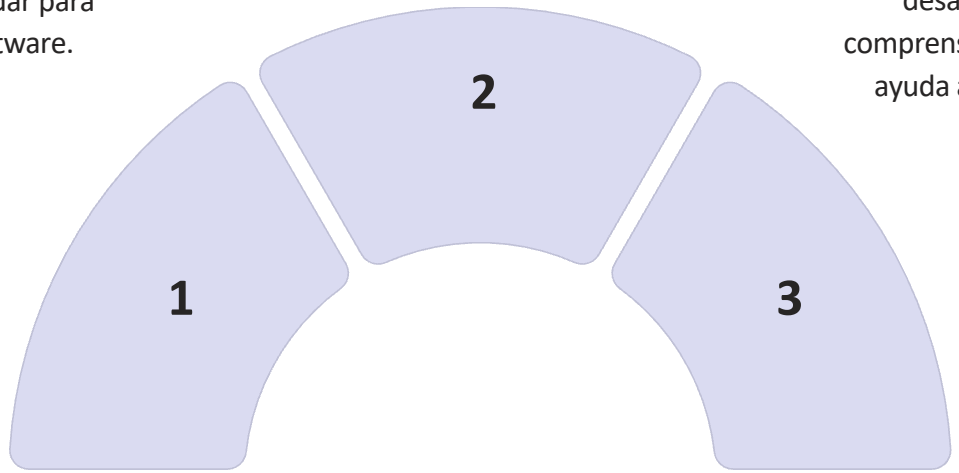
UML es un lenguaje estándar para modelar sistemas de software.

Tipos de Diagramas

UML ofrece una variedad de diagramas para diferentes propósitos, como casos de uso, secuencia y clases.

Beneficios

Mejora la comunicación entre desarrolladores, facilita la comprensión del sistema y ayuda a ayuda a identificar problemas.



Diagramas de Casos de Uso: Modelando la Interacción Usuario-Sistema

1

Objetivo

Los diagramas de casos de uso muestran cómo los usuarios interactúan con el sistema.

2

Componentes

Incluyen actores (usuarios o sistemas externos) y casos de uso (funciones del del sistema).

3

Beneficios

Ayudan a definir el alcance del sistema y a identificar las funcionalidades clave. clave.

Diagramas de Secuencia: Visualizando el Flujo de Procesos

1

Propósito

Los diagramas de secuencia muestran el flujo de mensajes entre objetos.

2

Estructura

Representan objetos en líneas de tiempo, con mensajes que indican la interacción entre ellos.

3

Aplicaciones

Se utilizan para comprender el comportamiento del sistema y para identificar posibles posibles problemas.

Diagramas de Clases: Estructurando la Arquitectura del Sistema

Función

Los diagramas de clases modelan la estructura del sistema y las relaciones entre entre clases.

Elementos

Muestran clases, atributos, atributos, métodos y relaciones como herencia, herencia, agregación y composición.

Beneficios

Facilitan la comprensión de la de la arquitectura del sistema sistema y ayudan a identificar identificar posibles puntos de puntos de mejora.

Integración de Requerimientos, Historias y UML en Proyectos Java

1

Planificación

Los requerimientos y las historias de usuario guían el desarrollo.

2

Diseño

Los diagramas UML ayudan a visualizar y modelar el sistema.

3

Implementación

El código se implementa en base a los requerimientos y el diseño.

4

Pruebas

Se asegura que el sistema cumple con los requerimientos.

UML

UML, siglas en inglés de "Unified Modeling Language" (Lenguaje de Modelado Unificado), es un lenguaje de modelado visual utilizado en el campo de la ingeniería de software para diseñar, especificar y documentar sistemas orientados a objetos.

El objetivo principal de UML es proporcionar a los profesionales de desarrollo de software un lenguaje común y comprensible para describir visualmente los aspectos estructurales y comportamentales de un sistema.

La creación de modelos que representan diferentes aspectos del sistema, como su estructura estática, comportamiento dinámico, interacciones entre sus componentes, y más.

DIAGRAMAS UML

Los diagramas son la principal herramienta de UML para representar estos modelos. Hay varios tipos de diagramas en UML, cada uno enfocado en aspectos específicos del sistema, como:

- **Diagrama de Clases:** Describe la estructura estática del sistema mostrando las clases, sus atributos, métodos y las relaciones entre ellas.

- **Diagrama de Casos de Uso:** Representa las interacciones entre el sistema y los actores externos, mostrando los diferentes casos de uso y cómo se relacionan entre sí.

- **Diagrama de Secuencia:** Ilustra cómo las diferentes partes del sistema interactúan entre sí a lo largo del tiempo, mostrando el orden de los mensajes intercambiados entre los objetos.

DIAGRAMAS UML

Diagrama de Estado: Muestra los posibles estados de un objeto y las transiciones entre ellos en respuesta a eventos específicos.

Diagrama de Actividad: Describe el flujo de actividades en un proceso o algoritmo, mostrando las acciones y decisiones que se toman en el camino.

Diagrama de Componentes: Representa las diferentes partes físicas del sistema y cómo se relacionan entre sí.

DIAGRAMAS DE CLASE EN JAVA

Los diagramas de clase en Java son una representación visual de las clases y las relaciones entre ellas en un sistema de software.

Los diagramas son una herramienta importante en el diseño orientado a objetos y se utilizan para visualizar la estructura estática de un sistema.

```java
public class Persona {
    // Atributos
    private String nombre;
    private int edad;

    // Constructor
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    // Métodos
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        this.edad = edad;
    }
}
```

```java
public class Estudiante extends Persona {
    // Atributos adicionales
    private String universidad;

    // Constructor
    public Estudiante(String nombre, int edad, String universidad) {
        super(nombre, edad);
        this.universidad = universidad;
    }

    // Método adicional
    public String getUniversidad() {
        return universidad;
    }

    public void setUniversidad(String universidad) {
        this.universidad = universidad;
    }
}
```

```
--------------------------------
|          Persona             |
--------------------------------
| -nombre: String              |
| -edad: int                   |
--------------------------------
| +Persona(nombre: String,     |
|          edad: int)          |
| +getNombre(): String         |
| +setNombre(nombre: String):  |
|          void                |
| +getEdad(): int              |
| +setEdad(edad: int): void    |
--------------------------------
```

```
--------------------------------
|          Estudiante          |
--------------------------------
| -universidad: String         |
--------------------------------
| +Estudiante(nombre: String,  |
|          edad: int,          |
|          universidad: String) |
| +getUniversidad(): String    |
| +setUniversidad(universidad: |
|          String): void       |
--------------------------------
```

IDE UML ONLINE

Lucidchart

Visual Paradigm Online

Microsoft Visio Online

magicdraw Architecture Made Simple

CONCLUSIONES Y PREGUNTAS